forsenClaw Design Document v5

A self-hosted multi-agent orchestration system with file-based agent definitions, clearance-stratified memory, OPA-based access control, and pub/sub-driven rooms. Runs on a NAS or similar always-on host.

Contents

1. Overview

forsenClaw is a self-hosted multi-agent orchestration system with file-based agent definitions, clearance-stratified memory, OPA-based access control, and pub/sub-driven rooms. It runs on a NAS or similar always-on host.

Users and agents are both **actors** in the system. The only structural difference: users authenticate with credentials; agents are defined by configuration files on disk and invoked by the system via Requests. Both participate in rooms, send messages, hold permissions, and are subject to clearance.

The design supports both **companion-style** agents (long-lived identity, accumulated memory, proactive behavior) and **task-style** agents (ephemeral, scoped context, reactive). These are configurations of the same agent primitive, controlled by per-agent feature flags.

1.1. Goals

- Persistent agent identities that accumulate memory across sessions.
- Clean separation between agent identity and the underlying model.
- Clearance as a context scope, not merely an access filter. The room's clearance level determines which memory strata the agent can see, forming a structural DLP boundary.
- Two orthogonal access axes — clearance (data) and permissions (actions) — enforced at distinct boundaries via BLP and OPA-backed ABAC.
- Multi-tier model routing per agent (primary / routine / sensitive).
- MCP as the universal tool integration surface.
- Protocol-driven rooms where protocols actively orchestrate agent invocation rather than passively gating sends.
- File-based agent definitions and memory, XDG-compliant.
- Self-hosted, single-user, sovereign over its own data. No required cloud dependency.
- Single binary. Target idle footprint: 50–80 MB.

1.2. Non-Goals

- Not a multi-tenant SaaS or cloud product.
- Not a replacement for coding harnesses; forsenClaw integrates them as MCP tool surfaces.
- Not a single-model chatbot wrapper.
- Not a competitor to general-purpose task orchestration frameworks (LangChain, AutoGen). forsenClaw's distinguishing feature is the unified companion + task model under a single permission and memory architecture.

1.3. Design Principles

Identity lives in memory, not in context Agents feel continuous because their memory files are continuous; the active context window is bounded and assembled per-invocation.

Clearance is a context scope, not a filter The room clearance determines which memory strata are present in the assembled context. An agent in a clearance-2 room does not have clearance-4 data available — it cannot leak what it cannot see. This is a structural DLP guarantee, not just an enforcement layer.

Defense in depth at the boundary, not the storage Memory is stored at full fidelity per stratum; clearance is enforced at assembly, retrieval, send, and context injection — not by deleting or summarizing data on disk.

Stateless models, stateful agents Models are compute primitives. Agents are the persistent identity that owns context, memory access, and permissions.

No ambient authority Every privileged action — tool dispatch, room creation, memory write, config change — is gated by an explicit permission grant evaluated by OPA.

Trust is at the agent, not the model An agent's clearance determines what data it may see. Within that clearance, the agent may route calls to any of its configured models.

Protocols orchestrate; agents respond Protocols actively issue Requests to agents. Agents do not decide when to speak — the protocol does. New interaction patterns are new protocols, not new primitives.

Files over databases for identity and knowledge Agent definitions and memory are files on disk — readable, editable, version-controllable without a running server. Databases are for queryable runtime state and audit.

Requests as the universal invocation primitive Room protocols, the proactive system, and event pub/sub all deliver work to agents through the same Request queue. There is no separate invocation path.

1.4. Tech Stack

Backend Go

- Concurrency: goroutines + channels.
- Database driver: standard `database/sql` with modernc.org/sqlite (pure-Go SQLite, no CGO).
- Migrations: goose or golang-migrate.
- Policy evaluation: embedded OPA (Open Policy Agent) with Rego.
- Scheduling: time.Ticker + priority queue for proactive ticks.

Frontend Vue 3 + TypeScript + Pinia

Storage

- Agent definitions: YAML files on disk.
- Agent memory: clearance-stratified Markdown files on disk (`MEMORY-k.md`, `memory/clearance-k/YYYY-MM-DD.md`).
- Room transcripts: JSONL files on disk.
- Room metadata, protocol state, request DAG, audit log: SQLite.
- Search index: SQLite (cache-tier, rebuildable).

Local inference Ollama over HTTP (not in-process).

2. Actor Model

An **actor** is any entity that participates in rooms, sends messages, and is subject to clearance and permissions. There are two kinds.

2.1. Users

A user authenticates with credentials (v1: username + password; later: hardware token, biometric). On authentication, the user receives a session token scoped to the connection. The user is the **root identity** — they can do anything: spawn agents, modify settings, grant or revoke permissions, read any audit log, terminate the server, read or write any memory layer or config file. Root access is identity, not a permission grant.

2.2. Agents

An agent is defined by a YAML configuration file on disk and invoked by the system via Requests. An agent has a role, model assignments, feature flags, clearance, and permissions. Agents do not authenticate — they are instantiated by the server from their file definitions.

2.3. Shared Attributes

Both users and agents:

- Participate in rooms as message senders and recipients.
- Are subject to room protocols (Request ordering, turn limits).
- Have a clearance level that governs data visibility and context assembly.
- Have permissions that govern actions (though users hold implicit root).
- Appear in room transcripts with their identity as the sender.

2.4. Differences

Concern	User	Agent
Authentication	Credentials	Definition file + Request invocation
Authority	Root (implicit)	Granted permissions
Memory ownership	N/A (user is the subject of memory)	Per-agent clearance-stratified files
Clearance	Implicitly top-tier	Configured per-agent
Lifecycle	Login/logout	Persistent or ephemeral
Invocation	Self-directed	Protocol issues a Request

This unification means the frontend treats DMs with agents and multi-participant rooms identically. A DM with the housewife is just a room with two participants. A structured debate room with three agents is the same primitive with a different protocol.

3. Agent Model

An **agent** is a persistent identity defined by:

1. A **definition file** (`agent.yaml`) — role, system prompt, behavioral configuration, model assignments, feature flags, clearance, permissions.
2. A **memory directory** — clearance-stratified `MEMORY-k.md` files for long-term memory, and `memory/clearance-k/YYYY-MM-DD.md` for daily notes.

The agent owns its context window assembly, its memory files, its permission set, and its identity across sessions. Models are stateless compute that the agent invokes.

3.1. Multi-Tier Model Assignment

Each agent declares three model tiers. Tiers describe **routing roles**, not trust boundaries — an agent's clearance governs what data it sees regardless of which tier handles the call.

`primary_model` High-reasoning tasks. Examples: Claude Sonnet, GPT-class.

`routine_model` Low-stakes ticks, proactive checks, summarization. Examples: local Gemma, local Llama.

`sensitive_model` The model for operations the user prefers to keep local or routed to a trusted provider. Typically local. Used by default for redaction proposals and other sensitivity-aware operations.

The agent decides which tier to invoke based on the task. This is not model fallback — tiers have different roles, not different quality levels.

3.2. Feature Flags

Flag	On	Off
`identity_continuity`	MEMORY-k.md files persist across sessions	Memory starts fresh each session
`daily_notes`	Daily note files maintained, indexed for search	No daily notes; context from MEMORY-k.md only
`proactive_triggers`	Participates in event-driven and scheduled Requests	Acts only when invoked via room protocol Request
`dreaming`	Background consolidation from daily notes to MEMORY-k.md	No automatic promotion; manual curation only
`rag`	Index worker starts; retrieval chunks may be injected	No embedding model, no index worker; RAGChunks stays nil

Companion-style agents typically have all flags on. Task-style agents typically have all flags off.

3.3. Context Assembly

Assembled fresh each invocation (per Request). The assembly differs by Request source.

3.3.1. Room Requests (Full Assembly)

Order, narrowing from background to immediate:

1. **System prompt** — from `agent.yaml` role description, plus a clearance notice injected by the assembler (see Section 5).
2. **MEMORY-k.md files** — all files where k ≤ room clearance, read in ascending order and concatenated. Additive: each file contains only what is new at that level.
3. **Daily notes** — today's and yesterday's notes for all clearance levels k ≤ room clearance, if `daily_notes=on`.
4. **Cross-room feed** — recent messages from all other rooms the agent participates in, labeled by room ID and merged chronologically. Filtered by the agent's clearance.
5. **Current room history** — windowed tail of the target room's transcript, starting at the room's `compacted_offset`.
6. **Turn budget notice** — remaining turns before user approval required.
7. **Request payload** — the actual message the agent is responding to, including any pending interjections.

The assembly narrows from ambient background context down to the immediate task. Current room history is placed adjacent to the request payload so recency bias works correctly for the turn being formed.

3.3.2. Event Requests (Minimal Assembly)

1. System prompt + clearance notice.
2. MEMORY-k.md files (all k ≤ agent's full clearance).
3. Daily notes (today and yesterday, all clearance levels).
4. Event payload.

No cross-room feed, no room history. Event Requests run in isolated context to prevent session contamination and keep inactivity timers honest.

3.3.3. Cross-Room Feed

At assembly time, query SQLite for all rooms where the agent participates, excluding the current room. Read the last `other_room_window` messages from each room, filter by clearance, merge chronologically, and format each message as `[#room-id --- relative-time] Sender: Content`. If the agent participates in no other rooms, this tier is empty.

3.3.4. Current Room History

Read only the tail of the current room transcript, starting from that room's `compacted_offset`. Tail reads seek from the end of the JSONL file and scan backward until the window is satisfied. This keeps the hot path bounded even for large rooms.

3.4. Agents Are Not Knowledge Specialists

LLMs already have broad knowledge. The role of a persistent agent is not "knows about cooking" or "knows about code" — it is defined by **what it knows about the user** (clearance), **what it can do** (permissions), and **what it's responsible for** (role). The housewife is the most trusted agent not because it knows the most about the world, but because it knows the most about **you**. A lower-clearance agent isn't dumber — it has less context about your life.

4. Memory Architecture

forsenClaw's memory is **files on disk** that agents read and write. The model only "remembers" what is saved to files — there is no hidden state.

Memory is **owned by agents, not rooms**. Each persistent agent has its own memory directory under `$XDG_DATA_HOME/forsenClaw/agents/<name>/`. Memory is organized in clearance strata: each file contains only information relevant to its level, and context assembly combines strata additively.

4.1. Clearance-Stratified Memory Files

Instead of a single `MEMORY.md`, each agent maintains a stack of files, one per clearance level that has content:

```
agents/housewife/
  MEMORY-1.md          ← public-facing role, communication style
  MEMORY-2.md          ← external-safe preferences, timezone
  MEMORY-3.md          ← active projects, professional context
  MEMORY-4.md          ← full personal context (dreaming target)
  MEMORY-5.md          ← vault: health, finances (manual only)
  memory/
    clearance-2/
      2026-05-23.md
    clearance-4/
      2026-05-23.md
```

```
clearance-5/
  2026-05-23.md
```

Files are additive, not duplicated. `MEMORY-3.md` contains only what is new at level 3, not a copy of levels 1 and 2. When assembling context at clearance 3, the assembler reads `MEMORY-1.md`, `MEMORY-2.md`, and `MEMORY-3.md` in order and concatenates them. The agent sees a coherent, unified memory at its current level.

Files are only created when there is content to write at that level. An agent that has never had a clearance-5 interaction will not have `MEMORY-5.md`.

All files are plain Markdown — human-editable, diffable, and git-trackable.

4.1.1. Memory Flow Rules

Writes happen at current clearance A memory write during a room interaction targets the stratum matching the room's current clearance level.

Data flows down via explicit redaction approval An agent wishing to pass sensitive content to a lower-clearance stratum generates a redaction proposal using its `sensitive_model`. The user reviews the original and proposed redaction side-by-side, optionally edits, and approves or rejects. Only on approval does the content cross the boundary.

Data flows up via dreaming or manual promotion The dreaming pass consolidates daily notes and may promote facts to a higher level (see Dreaming). Manual promotion: the user inspects a diff and instructs the agent to promote a fact. The agent writes to the higher stratum after the user approves.

Destructive operations require user confirmation Forgetting facts (deleting or overwriting established memory) requires user confirmation regardless of who issues the instruction. This prevents accidental or adversarial memory wipes.

4.1.2. Clearance-Level Granularity

Lower-clearance representations of a fact are not conflicts with higher-clearance representations — they are the same fact at different granularity. For example:

- `MEMORY-2.md`: "User works in tech."
- `MEMORY-4.md`: "User is a CS student at UNR working on a self-hosted multi-agent system called forsenClaw."

These coexist. The dreaming pass understands this and does not treat granularity differences as conflicts.

4.2. Daily Notes — Working Memory

Per-day Markdown files at `memory/clearance-k/YYYY-MM-DD.md`. Running context, observations, session summaries, detailed notes. Today's and yesterday's notes for all levels ≤ room clearance are loaded automatically into context.

Daily notes are the working layer. Agents write to them during sessions, capturing observations, decisions, and reasoning that may be useful later but is not yet curated into a `MEMORY-k.md` file.

Compaction summaries from older room transcripts are appended here so they fall outside the current-room window on later assemblies without mutating the transcript itself.

4.3. Dreaming — Background Consolidation

For agents with `dreaming=on`, a background pass periodically reviews daily notes and promotes durable material into the appropriate `MEMORY-k.md` file. This runs on the `routine_model` to keep costs low.

Dreaming runs at the agent's full clearance. It sees all strata and all daily notes. Conflict resolution:

- Higher-clearance fact + recency wins.
- Granularity differences (same fact at different levels) are not conflicts.
- Genuinely ambiguous conflicts (same level, different claims) are surfaced for user review rather than silently resolved.

Dreaming output defaults to level 4 (personal). Level 5 (vault) facts are never promoted automatically — they are always manually curated.

The dreaming pass writes a brief summary of what it promoted and why to a `DREAMS.md` file for human review.

4.4. Search Index

A SQLite-backed hybrid search index (vector similarity + keyword matching) over all `MEMORY-k.md` files and daily notes. Embeddings computed locally via Ollama.

The search index is infrastructure, not a memory layer. It is rebuildable from files on disk at any time and lives in `$XDG_CACHE_HOME/forsenClaw/`.

Retrieval is parameterized by the requesting agent's clearance level. An agent searches only memory strata at or below its current clearance.

With `rag: false` (the v1 default), no embedding model is loaded and no index worker starts.

4.5. Sessions and Lifecycle

A **session** is a bounded period of agent activity.

- Ends after 30 minutes of inactivity on a per-agent basis.
- Within a session: agents write observations to today's daily note at the appropriate clearance level.
- At session end: if there are uncompacted messages outside the guaranteed window, a compaction pass is scheduled. If `dreaming=on`, a consolidation pass may also run.
- Event Requests run in isolated session contexts — they do not extend or contaminate user-facing sessions.

4.6. Room Transcripts — Shared Conversation History

Room transcripts are stored as JSONL files at `$XDG_DATA_HOME/forsenClaw/rooms/<room-id>.jsonl`. Each line is a message record:

```
{
  "id": "msg_001",
  "timestamp": "2026-05-14T10:30:00Z",
  "room": "room_abc",
  "sender": "housewife",
  "clearance_tag": 4,
  "type": "message",
  "content": "..."
}
```

Transcripts are append-only and represent the shared record of what happened in a room, distinct from agent memory (an agent's personal understanding of what matters).

The active tail of each transcript is windowed by a per-agent, per-room compaction cursor stored in SQLite. Context assembly never re-injects messages before that cursor.

5. Access Control

Access control has two orthogonal axes with distinct enforcement points and mechanisms.

Clearance governs what an actor can **see** (data visibility). Enforced via Bell-LaPadula (BLP) rules applied during context assembly and retrieval.

Permissions govern what an actor can **do** (action capability). Enforced via Attribute-Based Access Control (ABAC) implemented with embedded OPA (Open Policy Agent) and Rego policies.

These axes are orthogonal. Operating at low clearance does not grant low-clearance tool access; permissions are evaluated independently.

5.1. Clearance — Context Scope

Clearance is a tiered data classification. Levels are totally ordered integers; higher numbers mean more trust and more sensitive data. The number of tiers and their descriptions are fully user-configurable in `hearth.yaml`.

Clearance is not merely an access filter — it is a context scope. The room's clearance level determines which memory strata the agent assembles. An agent operating in a clearance-2 room has only `MEMORY-1.md` and `MEMORY-2.md` in context. It cannot leak clearance-4 data because that data is not present. This is a structural DLP guarantee.

5.1.1. Default Five-Level Scheme

Level	Label	Description
1	public	Safe to expose anywhere. Role, communication style. Nothing personal.
2	external	Safe for external integrations. Preferences, timezone, things that inform external actions without revealing why.
3	professional	Task-scoped context. Active projects, professional context.
4	personal	Full personal context. Default landing zone for dreaming output.
5	private	Vault tier. Health, finances, anything where exposure causes real damage. Sparse, manually curated only.

Users can add, remove, renumber, or rename levels. The labels carry the semantics; the integers carry the ordering.

5.1.2. Clearance Notice Injection

Lower-clearance contexts are aware that higher-clearance data exists. The assembler injects a notice at the top of every context assembly:

```
You are operating at clearance level 2 (external). Higher-clearance
context exists but is not available in this context. If a question
requires deeper personal context, say so rather than guessing.
```

This prevents the agent from fabricating personal details it does not have access to.

5.1.3. Room Clearance Shifting

The user can shift a room's clearance up or down at any time. Shifting down is a deliberate "external-safe mode" for composing outbound content. The shift is the intentional boundary crossing — not a per-sentence approval. The frontend exposes a global clearance ceiling filter for session-wide scoping (see Section 11).

5.1.4. Bell-LaPadula Enforcement

BLP rules are enforced at three points:

1. **Context assembly** — only MEMORY-k.md and daily note files where k ≤ room clearance are included. Retrieval queries from the search index are filtered to the same ceiling.
2. **Outgoing message send** — the dispatcher checks a message's clearance tag against the destination room's ceiling. Rejected if above.
3. **Spawn-time context injection** — context passed to a child agent must not exceed the child's clearance. Dispatcher rejects violating spawns.

No read-up: agents cannot read data above their current clearance level.

No write-down: agents cannot write data below their current clearance level without explicit user approval (redaction proposal flow). See Section 4 for details.

Classification at write time: a message sent in a clearance-3 room is tagged clearance-3 in the transcript and never appears in a clearance-2 retrieval. Default classification is the sender's clearance, capped at the room's ceiling.

5.1.5. Soft Integrity Tagging (Soft Biba)

True Biba enforcement — no-read-down and no-write-up as hard integrity rules — is a v2 concern. v1 implements a lightweight integrity signal: when input arrives from a lower-clearance source in a higher-clearance context, the assembler annotates it:

```
[Source: clearance-2 (external) — treat with appropriate skepticism]
<content>
```

This makes provenance explicit without blocking the information. The annotation applies to:

- Messages in the transcript tagged below the current room ceiling.
- Tool results from external sources (clearance determined by the tool's declared level).
- Content forwarded from lower-clearance rooms via the cross-room feed.

The agent may act on the content but is aware it came from a less-trusted source and should not treat it as high-trust personal context.

Full Biba (hard no-read-down, write integrity labels, subject integrity levels) is deferred to v2.

5.1.6. Explicit Cross-Clearance Handoff

When an agent deliberately wants to pass sensitive content down — e.g., the housewife passing a redacted email summary to a clearance-2 integration agent:

```
propose_handoff(
    source_content: <high-clearance content>,
    target_clearance: <lower level>,
    destination: <agent or room>,
) -> proposed_redaction
```

The agent generates a proposed redaction using its `sensitive_model`. The dispatcher shows original and proposed redaction to the user side-by-side. User reviews, optionally edits, and approves or rejects. Only on approval does the redacted content cross the boundary.

5.2. Permissions — What an Actor Can Do

Permissions are fine-grained action capabilities evaluated by embedded OPA using Rego policies. Policy files live on disk at `$XDG_CONFIG_HOME/forsenClaw/policy.rego`, are git-trackable, human-editable, and diffable. Agents can propose policy changes via the config staging mechanic.

5.2.1. OPA Evaluation Model

The evaluation order is: **deny → require_confirmation → allow.**

- `deny` is unconditional and wins over all other rules. No specificity ambiguity.
- `require_confirmation` wins over `allow` when both apply (more restrictive takes precedence).
- Default deny everything not explicitly allowed.

5.2.2. Example Policy

```
package hearth.authz

default allow = false
default require_confirmation = false

# BLP: no write-down without approval
deny {
  input.action == "memory:write"
  input.target_clearance < input.room.clearance
}

# ABAC: email send allowed during business hours at clearance 2
allow {
  input.action == "tool:invoke"
  input.tool == "email:send"
  input.room.clearance <= 2
  input.agent.permissions[_] == "tool:invoke[email:send]"
  is_business_hours
}

# Require confirmation outside business hours
require_confirmation {
  input.action == "tool:invoke"
  input.tool == "email:send"
  not is_business_hours
}

is_business_hours {
  hour := time.clock(time.now_ns())[0]
  hour >= 9
  hour < 17
}
```

5.2.3. Permission Categories

- `tool:invoke[<tool_id>]` — MCP tool dispatch.

- `room:create`, `room:add_participant`, `room:close`, `room:extend_turn_limit`.
- `memory:write[<layer>]`, `memory:search[<scope>]` — write to own memory files, search across agents.
- `agent:spawn[<role>]`, `agent:terminate`, `agent:compact[<target>]`.
- `config:read[<scope>]`, `config:write[<scope>]` — read/write agent or server configuration files. Scopes: `self`, `server`, `agent:<name>`.
- `proactive:enable`, `proactive:act[<risk_tier>]`.
- `handoff:propose[<target_clearance>]`.
- `audit:read`.
- `policy:propose` — propose changes to the OPA policy file.

5.2.4. Default Effects for Config Permissions

Permission	Default Effect
`config:read[self]`	`allow`
`config:write[self]`	`require_confirmation`
`config:read[server]`	`require_confirmation`
`config:write[server]`	`require_confirmation`
`config:read[agent:*]`	`deny`
`config:write[agent:*]`	`deny`
`policy:propose`	`require_confirmation`

5.3. Config Staging and Approval

When an agent proposes a config change (writing to any file under `$XDG_CONFIG_HOME/forsenClaw/`), the flow is:

1. Agent writes the proposed new file to the staging area: `$XDG_DATA_HOME/forsenClaw/staging/agents/<name>/agent.yaml.proposed`.
2. OPA evaluates the write permission. If `require_confirmation`: the dispatcher computes a diff and surfaces it to the user for review. The user approves, edits, or rejects.
3. If the effect is `allow`: the system applies the change directly.
4. If the effect is `deny`: the agent cannot propose the change.

On approval, the proposed file replaces the config file. On rejection or expiry, the proposed file is deleted. The audit log records the proposal, diff, and outcome.

One pending proposal per agent per config file. A new proposal overwrites the previous one.

Policy file proposals follow the same staging flow with `policy:propose`.

5.4. Audit

The audit log records **actions taken** and **attempts denied**:

- Tool invocations (call + arguments + result status).
- Permission denials and OPA evaluation results.
- `require_confirmation` checks (request + user response).
- Config proposals (diff + outcome).
- Agent spawn and teardown (ephemeral agent definitions snapshotted here).
- Cross-clearance rejections.

- Handoff proposals and resolutions.
- Request DAG traversal (full proof trace; see Section 6).

Append-only, SQLite-backed, accessible to the user without restriction. `audit:read` is its own permission for agents; they do not get it by default.

6. Rooms

Rooms are clearance-bounded conversation spaces. Their primary role is information architecture — defining what an agent can see — not orchestration. A room is a context isolation unit.

6.1. Room Structure

A room has:

- A **participant list** (actors — users and/or agents).
- A **transcript file** (JSONL on disk).
- A **clearance ceiling** — the highest clearance level any message may carry, and the scope at which agents assemble context when operating here.
- A **parent** (optional) — for structural grouping under a project.

Room metadata (participant list, clearance ceiling) is stored in SQLite. The transcript itself is the JSONL file.

6.2. Clearance Ceiling as Confidentiality Boundary

The clearance ceiling is the room's defining property. It determines:

- Which `MEMORY-k.md` strata agents assemble (k ≤ ceiling).
- Which daily notes are in scope.
- The classification tag on messages written here: `min(sender.clearance, room.ceiling)`.
- Which messages from this room surface in other agents' cross-room feeds.

An agent cannot leak what it cannot see. A clearance-2 room agent has only clearance-1 and clearance-2 memory assembled. Higher-clearance data is structurally absent — not filtered at query time, simply not present. This is the structural DLP guarantee described in Section 5.

A user operating in a low-clearance room can compose outbound content without risk of contaminating it with personal detail. The room ceiling is the intentional boundary crossing — not a per-message filter, not a classification warning, but a structural constraint on what enters the context window.

6.2.1. Room Clearance Shifting

The user can change a room's clearance ceiling at any time. Shifting down is a deliberate "external-safe mode." Shifting up admits higher-clearance context on the next assembly. The frontend exposes a global ceiling filter for session-wide scoping (see Section 11).

6.2.2. Soft Integrity Tagging

When an agent in a higher-clearance context receives input from a lower-clearance source — a message tagged below the room ceiling, a tool result from an external source, content forwarded from a lower-clearance room — the assembler annotates it with a trust label:

```
[Source: clearance-2 (external) — treat with appropriate skepticism]
<content>
```

This is a **soft Biba** integrity signal. It does not block the information but makes provenance explicit so the agent applies appropriate skepticism. Lower-clearance sources may carry unverified, publicly-sourced, or externally-derived content that should not be treated as high-trust personal context.

Full Biba enforcement (no-read-down and no-write-up as hard rules, integrity labels on subjects) is a v2 concern. The v1 soft model establishes the convention without the hard enforcement machinery.

6.3. Room Creation

Any actor with the `room:create` permission may create a room. The creator selects participants and clearance ceiling.

A participant whose clearance exceeds the room ceiling cannot join — their context would be constrained without their awareness. The correct pattern is to create a room at the appropriate ceiling and invite participants whose clearance is at or below it.

6.4. Invocation

When any message is sent to a room, every agent participant receives a Request. There is no protocol layer governing turn order — all agents are invoked on every message. Room behavior is defined by the clearance scope, not orchestration rules.

6.5. Requests — Universal Invocation Primitive

A **Request** is the single invocation primitive. Room messages, the proactive system, and pub/sub event triggers all deliver work to agents through Requests.

6.5.1. Request Structure

```
type Request struct {
    ID       string
    Room     string        // empty for system and event requests
    Target   string        // agent name
    Source   RequestSource // room | system | event
    Payload  RequestPayload
    Deadline time.Time     // optional timeout
    ParentID string        // set when this request is a child in the DAG
}

type RequestPayload struct {
    Messages  []Message
    Metadata  map[string]any
}
```

Request source determines context assembly:

Source	Context Assembly
`room`	Full assembly: system prompt + clearance notice + memory strata + daily notes + cross-room feed + current room history + request payload
`system`	Minimal: system prompt + memory strata + event payload
`event`	Minimal: system prompt + memory strata + event payload

6.5.2. Dependency DAG

Each agent maintains a **dependency DAG** of its pending Requests. This unifies all blocking operations under a single model.

Each Request is a node. A directed edge from A → B means "A is blocked waiting for B to resolve." When B resolves, A is unblocked and resumes with B's response injected into context.

```
type RequestNode struct {
    Request   Request
    State     NodeState  // pending | in_progress | blocked | resolved | failed
    BlockedBy []string   // IDs of Requests this node is waiting on
    Children  []string   // IDs of child Requests issued by this node
}
```

The DAG unifies all blocking operations:

Operation	DAG representation
Agent-to-agent delegation	Edge to child Request node
Tool call requiring confirmation	Edge to user approval node
`require_confirmation` OPA result	Edge to user approval node

Cycle detection is required and runs before any edge is committed. The dispatcher performs a DFS from the new edge's target back to the source before adding the edge; a detected cycle causes the child Request to be rejected.

The audit log records the DAG as a traversal, providing a proof trace of every delegation, tool call, confirmation, and final output for each Request.

6.6. Turn Limits

All rooms enforce a turn limit on agent-to-agent exchanges without user participation. A **turn** is one completed Request response. Turn limits are per-room and apply only when no user has sent a message recently; a user message resets the counter.

Each room has configurable soft and hard limits. When the hard limit is reached, the system pings the user via `room:extend_turn_limit`. The agent sees the budget in assembled context:

```
[Turn budget: 4 of 6 responses used. After 2 more, user approval required.]
```

6.7. Projects

A **project** is a structural grouping over rooms. A project has:

- A name and description.
- A set of child rooms.
- An optional set of associated agents.
- A clearance ceiling inherited by child rooms unless overridden.
- A summary view that surfaces major events from child rooms.

Projects are navigation and organizational primitives, not permission boundaries.

7. Agent Lifecycle

7.1. Persistent Agents

Long-lived. Defined by files on disk, exist indefinitely. Stable identity, accumulated memory, long-term permission grants. The user typically has 1–3.

A persistent agent has:

- A definition file at `$XDG_CONFIG_HOME/forsenClaw/agents/<name>/agent.yaml`.
- A memory directory at `$XDG_DATA_HOME/forsenClaw/agents/<name>/` with clearance-stratified `MEMORY-k.md` files and `memory/clearance-k/` daily note subdirectories.

The housewife is the default persistent agent — highest clearance, broadest permissions, most accumulated context about the user. It is not an "orchestrator" architecturally — it's the most trusted agent, and it's good at orchestration as a consequence of knowing the most. Users are not required to have an orchestrator; they can talk directly to any agent.

Persistent agents may not self-terminate without user confirmation — loss of accumulated memory context is irreversible.

7.2. Ephemeral Agents

Spawned for a task, given scoped context, terminated on completion. Ephemeral agents exist as in-memory objects with a captured configuration. **Ephemerals are not a security primitive** — clearance and OPA handle access scoping. Ephemerals exist for parallelism and task isolation only.

Characteristics:

- Spawned by a persistent agent or the user.
- Inherit a subset of the spawner's permissions, never more.
- Clearance level equal to or lower than the spawner's.
- Scoped context injection at spawn (not a full memory dump).
- Do not accumulate memory — no `MEMORY-k.md`, no daily notes, no files on disk.
- Discarded after task completion, timeout, room closure, or revocation.
- Definition snapshotted into the audit log at spawn time so references are never broken.

Ephemeral agents are where bulk work happens. Most tasks need a focused context, a defined goal, and teardown — not a long-lived identity.

7.3. Spawn

The spawner must hold `agent:spawn[<role>]`. The spawner supplies:

- Task description.
- Context injection (dispatcher validates against child's clearance).
- Permission set (subset of spawner's).
- Timeout or termination condition.
- Whether the agent joins an existing room or gets a new one.

The ephemeral agent's definition is snapshotted into the audit log at spawn time.

7.4. Teardown

Triggers: task completion, timeout, room closure, revocation by parent or user.

On teardown:

- Room transcript already has the full conversation (JSONL file).
- Definition snapshot already in audit log (from spawn).
- In-memory agent state is discarded.
- Audit entry records termination cause.

7.5. Compaction

A persistent agent (typically housewife) may trigger compaction of another persistent agent, subject to `agent:compact[<target>]`. Compaction is a housekeeping pass that summarizes old room transcript messages into the agent's daily note at the appropriate clearance level, then advances the per-agent, per-room `compacted_offset` cursor in SQLite.

Compaction is triggered when either of these happens:

- The assembled context exceeds `context.compaction_trigger`.
- The agent session ends after 30 minutes of inactivity.

7.5.1. Batch Sizing

The batch size is dynamic. On a byte-threshold trigger, forsenClaw walks forward from `compacted_offset`, accumulating message sizes until the removed bytes are enough to bring the assembled context down toward `context.compaction_target`. That batch is capped so compaction never crosses the guaranteed tail window (`context.minimum_guaranteed`). If the remaining uncompacted tail is smaller than `min_compaction_batch`, compaction is skipped.

Session-end compaction compacts all messages outside the guaranteed window, regardless of byte size. If even compacting everything outside the guaranteed window still leaves the context above target, forsenClaw proceeds anyway. The floor is hard; the model handles the larger context.

7.5.2. Compaction Request Flow

Before a real Request is assembled, the dispatcher may issue a system compaction Request that tells the agent to summarize a specific room/message range into its daily note. When compaction succeeds, the cursor advances to the end of the compacted batch and the real Request is assembled afterward. If compaction fails (model error, timeout), the real Request still proceeds with the uncompacted context.

7.6. SLM Harness Considerations

The invocation pipeline is the correct layer for small-model scaffolding. Techniques for SLMs (per-turn skill injection, output repair for malformed tool calls, quality monitoring for empty/looping responses, thinking-budget caps) apply at the Request processing level, transparent to the agent definition and protocol.

8. Storage Architecture

forsenClaw follows the XDG Base Directory Specification. All paths respect environment variable overrides (`$XDG_CONFIG_HOME`, `$XDG_DATA_HOME`, `$XDG_CACHE_HOME`).

8.1. Directory Layout

```
$XDG_CONFIG_HOME/forsenClaw/              # Config — small, version-controllable
├── hearth.yaml                           # Server config: providers, models, listen
addr
├── policy.rego                           # OPA policy file (ABAC rules)
└── agents/
```

```
    ├── housewife/
    │   └── agent.yaml                    # Role, models, flags, permissions,
clearance
    └── scout/
        └── agent.yaml

$XDG_DATA_HOME/forsenClaw/               # Data — large, persistent
├── agents/
│   ├── housewife/
│   │   ├── MEMORY-1.md                  # Public-facing facts (clearance 1)
│   │   ├── MEMORY-2.md                  # External-safe facts (clearance 2)
│   │   ├── MEMORY-3.md                  # Professional context (clearance 3)
│   │   ├── MEMORY-4.md                  # Full personal context (clearance 4)
│   │   ├── MEMORY-5.md                  # Vault (clearance 5, manual only)
│   │   ├── DREAMS.md                    # Dreaming activity log (optional)
│   │   └── memory/
│   │       ├── clearance-2/
│   │       │   └── 2026-05-23.md
│   │       ├── clearance-4/
│   │       │   └── 2026-05-23.md
│   │       └── clearance-5/
│   │           └── 2026-05-23.md
│   └── scout/
│       ├── MEMORY-1.md
│       ├── MEMORY-2.md
│       └── memory/
│           └── clearance-2/
│               └── 2026-05-23.md
├── rooms/
│   ├── <room-id>.jsonl                  # Room transcripts
│   └── ...
├── staging/                             # Pending config and policy proposals
│   └── agents/
│       └── housewife/
│           └── agent.yaml.proposed
└── db/
    ├── rooms.db                         # Room metadata, protocol state, compaction
cursors
    └── audit.db                         # Audit log

$XDG_CACHE_HOME/forsenClaw/              # Cache — rebuildable
├── search.db                            # Hybrid search index
└── embeddings/                          # Cached embedding vectors
```

8.2. Server Configuration

`hearth.yaml` includes a `context:` block:

```yaml
context:
  current_room_window: 50
  other_room_window: 10
  compaction_trigger: 524288
  compaction_target: 262144
  minimum_guaranteed: 20
```

- `current_room_window`: guaranteed minimum tail size for the active room.
- `other_room_window`: number of recent messages read from each other room.
- `compaction_trigger`: assembled-context byte threshold that triggers compaction.
- `compaction_target`: desired size after compaction.
- `minimum_guaranteed`: hard floor that compaction never crosses.

Invalid values are rejected at startup. In particular, `compaction_target` must be lower than `compaction_trigger`.

Clearance level labels are also defined in `hearth.yaml`:

```yaml
clearance_levels:
  - level: 1
    label: public
    description: "Safe to expose anywhere."
  - level: 2
    label: external
    description: "Safe for external integrations."
  - level: 3
    label: professional
    description: "Task-scoped context."
  - level: 4
    label: personal
    description: "Full personal context."
  - level: 5
    label: private
    description: "Vault tier. Manual curation only."
```

8.3. What Lives Where and Why

Data	Location	Rationale
Agent definitions	Config	Identity. Version-controllable. Mountable in Docker.
OPA policy	Config	Git-trackable, diffable, human-editable. Proposed via staging.
Server config	Config	Providers, models, clearance labels. Rarely changes.
Agent memory (MEMORY-k.md)	Data	Agent-written, grows over time, persistent.
Daily notes	Data	Clearance-stratified; grows over time.
Room transcripts	Data	Append-only conversation records.
Config proposals	Data	Staging area for pending changes. Not yet config.
Room metadata	Data (SQLite)	Dynamic, queryable (list rooms, filter by participant).
Compaction cursors	Data (SQLite)	Per-agent, per-room `compacted_offset`.
Request DAG	Data (SQLite)	Node and edge state for in-flight Requests.
Audit log	Data (SQLite)	Structured queries, append-only guarantees, DAG traces.

Data	Location	Rationale
Search index	Cache	Rebuildable from files. Excludable from backups.
Embeddings	Cache	Rebuildable. Expensive to recompute but not critical.

8.4. SQLite Schemas

8.4.1. Compaction Cursors (rooms.db)

```
CREATE TABLE IF NOT EXISTS compaction_cursors (
  agent_name TEXT NOT NULL,
  room_id    TEXT NOT NULL,
  compacted_offset INTEGER NOT NULL DEFAULT 0,
  updated_at DATETIME NOT NULL,
  PRIMARY KEY (agent_name, room_id)
);
```

8.4.2. Request DAG (rooms.db)

```
CREATE TABLE IF NOT EXISTS request_nodes (
  id        TEXT PRIMARY KEY,
  room_id   TEXT,
  target    TEXT NOT NULL,
  source    TEXT NOT NULL,  -- room | system | event
  state     TEXT NOT NULL,  -- pending | in_progress | blocked | resolved | failed
  parent_id TEXT,
  payload   TEXT NOT NULL,  -- JSON
  created_at DATETIME NOT NULL,
  resolved_at DATETIME
);

CREATE TABLE IF NOT EXISTS request_edges (
  parent_id TEXT NOT NULL,
  child_id  TEXT NOT NULL,
  PRIMARY KEY (parent_id, child_id),
  FOREIGN KEY (parent_id) REFERENCES request_nodes(id),
  FOREIGN KEY (child_id)  REFERENCES request_nodes(id)
);
```

8.5. Version Control

`$XDG_CONFIG_HOME/forsenClaw/` can be configured as a git repo containing agent definitions, server config, and the OPA policy file. `$XDG_DATA_HOME/forsenClaw/agents/` is optionally a second repo for memory history. Room transcripts can be included or excluded; they are append-only so git handles them, but they grow.

The `db/` directory is excluded from version control. SQLite databases are queryable runtime state; audit can be exported if archival is needed.

8.6. Docker Volume Mapping

```
volumes:
  - forsenClaw_config:/home/user/.config/forsenClaw   # Small, version-controllable
```

```
  - forsenClaw_data:/home/user/.local/share/forsenClaw # Large, persistent
  # cache intentionally omitted — rebuilt on start
```

Config volume can be mounted read-only to prevent agent self-modification entirely (overriding any `config:write` permissions). This is a deployment-level decision.

9. Proactive Action System

Agents with `proactive_triggers=on` participate in a proactive loop. The proactive system is unified with the general invocation pipeline: all proactive work is delivered as **event Requests** via pub/sub. There is no separate invocation path for proactive behavior.

9.1. Pub/Sub and Event Requests

External signals are delivered via a pub/sub bus. Trigger sources subscribe to topics; when a topic fires, the system issues an event Request to the target agent.

Event Request context assembly is minimal: system prompt + memory strata + event payload (see Section 3). No cross-room feed, no room history. This keeps proactive invocations isolated from active sessions.

9.2. Triggers

9.2.1. Interrupt Triggers

Event-driven. An external signal generates an event Request:

- Incoming email matching a filter.
- Calendar event approaching a threshold time.
- Webhook from an external service.
- Time threshold (e.g., “it has been 48 hours since last check”).

The dispatcher subscribes to the pub/sub bus on behalf of agents with `proactive_triggers=on`. When a message arrives on a subscribed topic, the dispatcher constructs an event Request and enqueues it for the target agent.

9.2.2. Scheduled Triggers

Agent-driven. At the end of each check, the agent decides when to next run and writes a scheduled event (cron syntax or duration offset). When the schedule fires, the system issues an event Request.

Both trigger types arrive at the agent through the same Request queue as room Requests. The agent’s processing pipeline is identical in both cases.

9.3. Risk Tiers

Risk tier is a property of the action’s permission definition, not the agent’s judgment. The OPA policy evaluates it; the dispatcher enforces it.

Risk Tier	Policy
Low	Cosmetic, reversible, local. Routine model may act directly. OPA: `allow`.
Medium	Observable effects, messages other actors. Primary model required. OPA: `allow` with logging.

Risk Tier	Policy
High	Irreversible or external side effects. OPA: `require_confirmation`. User confirmation required before the action proceeds.

High-risk proactive actions create a blocked Request node in the DAG. The block resolves when the user approves or denies. On approval the action executes; on denial the Request is marked failed and the agent is notified.

9.4. Confidence Gating

Each proactive decision is annotated with a confidence score by the agent. Below a configurable threshold, the agent surfaces a suggestion rather than acting. This is independent of risk tier — a low-risk, low-confidence action surfaces as a suggestion rather than executing silently.

9.5. Context Isolation

Event Requests run in isolated context, separate from any active room session. Each invocation assembles fresh context (memory strata + daily notes + event payload). Proactive ticks do not extend or contaminate user-facing session inactivity timers.

9.6. Notification Delivery

Proactive outputs are delivered as messages in the appropriate room, or as system notifications for high-priority items. There is no separate "inbox" surface — the room the agent belongs to (or a default notification room) is where proactive outputs land. The frontend surfaces unread counts and notification badges per room.

10. MCP Integration

MCP (Model Context Protocol) is the universal tool integration surface. All external capabilities — email, calendar, file access, code execution, web search, system management, knowledge bases — are exposed as MCP tool servers.

10.1. Architecture

MCP servers run as separate processes (or remote services) and register with forsenClaw's tool registry. Each server exposes a set of tools with schemas. forsenClaw acts as the MCP client — the dispatcher invokes tools on behalf of agents.

10.2. Tool Injection

v1: static injection. At invocation time, all MCP tool schemas the agent is permitted to use (per its `tool:invoke[...]` permissions, evaluated by OPA) are resolved and injected into the API call. Simple and correct.

v2 candidate: dynamic injection. For agents with broad permissions and many available tools, a relevance filter matches the Request payload against tool descriptions and injects only top-K relevant schemas. Reduces context bloat without changing the agent or permission model.

10.3. Tool Routing

Tools may be hosted in two locations:

1. **On the server** — built-in MCP servers (web search, system tools). Always reachable.
2. **Remote services** — third-party MCP servers accessed over the network.

If a tool's host is unreachable, the call fails — no fallback.

10.4. Knowledge Base

The knowledge base is an MCP tool surface, not a custom forsenClaw primitive. Options:

- An existing knowledge-base solution (Obsidian + MCP adapter, dedicated RAG service) exposed as MCP tools.
- A simple forsenClaw-hosted document store with MCP tools — if no external solution fits.

Agents interact with the knowledge base through normal tool invocation, subject to OPA permission evaluation.

10.5. Permission Integration

MCP tool invocation is gated by `tool:invoke[<tool_id>]` permissions evaluated by OPA. The OPA policy receives the full invocation context — agent identity, room clearance, tool ID, arguments, time-of-day, and any other relevant attributes — and returns `allow`, `require_confirmation`, or `deny`.

An agent permitted `tool:invoke[email:*]` with `require_confirmation` for `tool:invoke[email:send]` can read email freely but needs user approval to send. This confirmation creates a blocked node in the Request DAG (see Section 6) that resolves on user response.

OPA evaluation runs in the dispatch layer **before** the executor is invoked. The executor itself is a pure transport layer: it resolves the tool, converts arguments, calls the MCP client, and records the audit event. It assumes OPA has already approved the call. This separation keeps the executor stateless and testable independently of the policy engine.

10.6. DLP Boundary

The MCP tool integration surface is bounded by the current room clearance. Tools can only exfiltrate data present in the assembled context. Since the context is assembled at the room's clearance level, higher-clearance data is structurally absent — it cannot be leaked even if a tool call attempts to transmit it. This is the structural DLP guarantee described in Section 5.

11. Frontend

forsenClaw ships a Vue 3 + TypeScript + Pinia web frontend served by the Go binary. The frontend communicates with the backend via WebSocket for real-time updates and REST for state queries.

11.1. Views

Room list All rooms the user participates in. Shows unread count and notification badges. Grouped by project if applicable.

Room view The active transcript, message composer, participant list, and clearance ceiling control. Streaming agent responses rendered in real time.

Agent config viewer Read-only view of an agent's `agent.yaml`. Diff view for staged config proposals awaiting approval.

Settings Server config (`hearth.yaml`), model provider registry, clearance level label customization.

Audit log viewer Filterable list of audit events. DAG visualization for individual Requests.

11.2. Clearance Ceiling Filter

A global clearance ceiling filter allows the user to set a session-wide ceiling (e.g., clearance 2 at work). When active:

- Messages tagged above the ceiling are hidden from the transcript view.
- Memory views show only strata ≤ the ceiling.
- The active room's clearance is constrained to ≤ the ceiling.
- Agent context assembly respects the lower ceiling for the duration of the session.

The ceiling filter is a session UI setting — it does not mutate stored data. Dropping the filter restores full visibility.

11.3. Room Clearance Shift

Each room view exposes a clearance ceiling control. The user can shift the room's clearance up or down at any time. This is the primary mechanism for "external-safe mode" when composing content destined for external integrations.

Shifting down is a deliberate, explicit boundary crossing. The intent is that the act of shifting is the approval — not a per-sentence confirm dialog.

The current clearance level is always visible in the room UI so the user knows what context the agent has available.

11.4. Request DAG Visualization

The audit log viewer includes a DAG visualization for individual Requests. Each node shows:

- Request ID, source (room/system/event), target agent.
- Current state (pending / in_progress / blocked / resolved / failed).
- Blocked-by edges to child Requests or approval nodes.
- Timestamps and resolution outcomes.

This provides a full proof trace for debugging agent behavior, reviewing delegations, and auditing tool use.

11.5. Config Proposal Diff View

When an agent proposes a config change, the frontend surfaces a diff review panel showing the current file and proposed file side-by-side. The user can:

- Approve the proposal as-is.
- Edit the proposed file inline before approving.
- Reject the proposal.

The same panel is used for OPA policy proposals.

12. Appendix

12.1. Agent Configuration Examples

12.1.1. The Housewife — Most Trusted Persistent Agent

```
name: housewife
role_description: >
  Personal agent. Manages household, calendar, meal planning, coordinates
  other agents for projects. Highest clearance, broad permissions. Acts
  proactively on routine matters; surfaces high-risk decisions to the user.

models:
```

```
  primary: gemma-4-local
  routine: gemma-4-local
  sensitive: gemma-4-local

feature_flags:
  identity_continuity: true
  daily_notes: true
  proactive_triggers: true
  dreaming: true

clearance: 5

permissions:
  - tool:invoke[*]
  - room:create
  - room:add_participant
  - room:extend_turn_limit
  - memory:write[*]
  - memory:search[*]
  - agent:spawn[*]
  - agent:terminate
  - agent:compact[*]
  - config:read[self]
  - config:write[self]
  - config:read[server]
  - config:write[server]
  - proactive:enable
  - proactive:act[low, medium]
  - handoff:propose[*]
  - policy:propose
  - permission:grant  # condition: granted <= self
```

12.1.2. Scout — Lower-Clearance External Agent

```
name: scout
role_description: >
  Handles external-facing tasks: web searches, API calls, public data
  retrieval. Lower clearance; assembles only external-safe context. Good
  for tasks where data leaves the system.

models:
  primary: claude-sonnet-4.6
  routine: gemma-4-local
  sensitive: gemma-4-local

feature_flags:
  identity_continuity: true
  daily_notes: true
  proactive_triggers: false
  dreaming: true

clearance: 2

permissions:
```

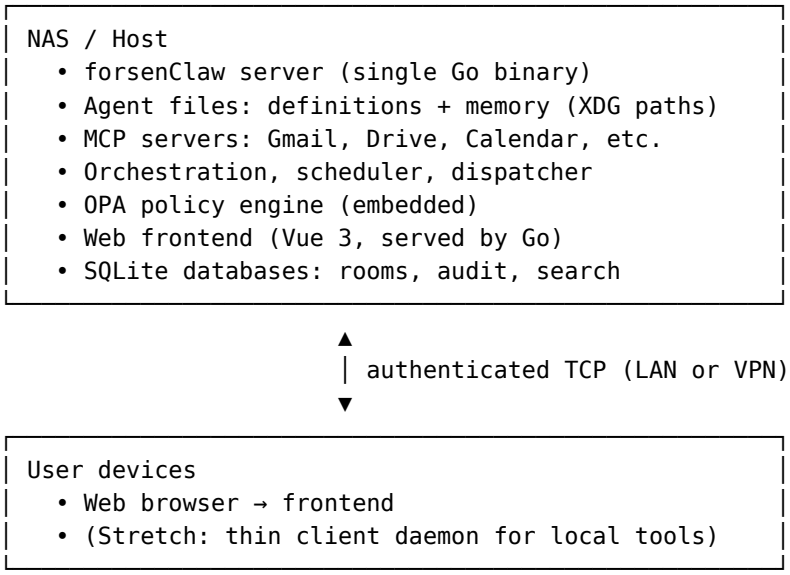
```
  - tool:invoke[web:*, api:*]
  - room:create
  - memory:write[self]
  - memory:search[self]
  - config:read[self]
```

12.1.3. Ephemeral Task Agent

In-memory only, definition snapshotted to audit at spawn time.

```
name: code_review_<task_id>
role_description: >
  Review the diff at <path>. Report findings. Terminate.

models:
  primary: claude-sonnet-4.6
  routine: gemma-4-local
  sensitive: gemma-4-local

feature_flags:
  identity_continuity: false
  daily_notes: false
  proactive_triggers: false
  dreaming: false

clearance: 3

permissions:
  - tool:invoke[code:read_file, code:review_diff]
  - memory:search[self]

timeout: 600s
parent: housewife
```

12.2. Network Topology



Every connection is token-authenticated regardless of network. LAN is convenient, not trusted. Tokens are scoped per client and revocable from the server.

12.3. Model Provider Registry

All model calls go through a provider registry defined in `hearth.yaml`.

```yaml
providers:
  - name: anthropic
    protocol: anthropic
    base_url: https://api.anthropic.com
    api_key: "${ANTHROPIC_API_KEY}"

  - name: ollama
    protocol: openai_compatible
    base_url: http://ollama-docker-container-name:11434
    # tool_mode: xml  # use "xml" for local models with no native tool support

models:
  claude-sonnet-4.6:
    provider: anthropic
    provider_model: claude-sonnet-4-20250514

  gemma-4-local:
    provider: ollama
    provider_model: gemma3:12b
```

- Agents reference model strings (`claude-sonnet-4.6`, `gemma-4-local`).
- The registry resolves to provider + provider-specific model ID.
- Inference uses a single `Infer(...)` entrypoint dispatching to a protocol adapter.
- Streaming is first-class: adapters yield normalized chunks.
- API keys resolved from environment variables, never stored in config files.

12.4. v1 Scope and Phasing

12.4.1. v1 — Usable System

The minimum viable forsenClaw: a single-user system where you can chat with persistent agents who remember things, use MCP tools, and manage rooms.

Must have:

- Actor model (user login + file-based agent definitions).
- Persistent and ephemeral agents with feature flags.
- Clearance-stratified memory: MEMORY-k.md + clearance-k daily notes.
- Rooms with FreeForm protocol (2 participants).
- Request-based invocation, dispatcher, context assembly.
- Cross-room feed plus bounded current-room tail in context assembly.
- Tail reads from transcript end, keyed by per-agent, per-room compaction cursor.
- Compaction Request flow and `compacted_offset` persistence.
- BLP enforcement (no read-up, no write-down) at assembly, send, spawn.
- OPA-backed ABAC for permissions; default policy file shipped with binary.
- MCP tool integration with static schema injection.

- Model provider registry (Anthropic + Ollama) in `hearth.yaml`.
- Request DAG: node/edge tracking, cycle detection, audit integration.
- Room metadata and audit in SQLite. Transcripts as JSONL.
- Web frontend: room list, room view, DMs, agent config viewer, settings, clearance ceiling filter, room clearance shift control.
- Audit log with DAG traversal view.
- Single Go binary deployment.
- XDG-compliant storage layout.

Nice to have in v1:

- RoundRobin and FireAndForget protocols.
- Broadcast protocol.
- Projects as structural grouping.
- Proactive event Requests via pub/sub.
- Basic notification delivery (in-room + badge counts).
- Config and policy diff review UI.
- Dreaming pass.

Deferred to v2:

- IterativeDraft protocol.
- Custom protocol plugin interface.
- Dynamic MCP tool injection (relevance filtering).
- Client daemon for local tools.
- Knowledge base MCP server.
- Push notifications.
- Cost accounting per agent/room.
- Memory budget management (auto-truncation of MEMORY-k.md).
- RAG-enabled context injection.
- Export/import room + agent bundles.

12.4.2. Build Order

1. **Storage layout + server config.** Establish XDG paths, `hearth.yaml` parsing, clearance level config, agent definition loading from disk.
2. **Model provider registry + inference adapter.** Get Ollama and Anthropic calls working. Everything else depends on this.
3. **Agent definitions + clearance-stratified memory.** MEMORY-k.md, daily notes per clearance level, search index. Test with a single agent reading and writing its own memory.
4. **Rooms + FreeForm protocol + Request dispatcher.** The messaging backbone. Room metadata in SQLite, transcripts as JSONL. User can chat with an agent.
5. **BLP enforcement + OPA permission evaluation.** Enforce at the three boundary points. Add audit logging. Implement config proposal and approval flow.
6. **MCP integration.** Register MCP servers, inject tool schemas, route calls. Test with a real MCP server (e.g., filesystem).
7. **Request DAG.** Node/edge tracking, cycle detection, blocked-node resolution, audit DAG trace.
8. **Session lifecycle + dreaming.** Session end triggers, dreaming pass from daily notes → MEMORY-k.md at appropriate level.

9. **Frontend.** Room list, room view, agent config, settings, clearance ceiling filter, room clearance shift, config diff review. Connect via WebSocket.
10. **Additional protocols.** RoundRobin, Broadcast, FireAndForget. Test structured debate.
11. **Projects.** Grouping, summary view, child room management.
12. **Proactive system.** Event Requests via pub/sub, scheduled triggers, risk tiers, confidence gating.

12.5. Open Questions

12.5.1. Search Index Location

Should the search index live in `$XDG_CACHE_HOME` (rebuildable, excluded from backups) or `$XDG_DATA_HOME/db/` (persistent)? Rebuilding the index requires re-embedding all memory files, which costs time and compute. If the embedding model changes, the index must be rebuilt anyway. Initial approach: cache, with a rebuild command (`forsenClaw index --rebuild`).

12.5.2. Dreaming Quality

Dreaming effectiveness depends on the routine model's ability to judge what is durable vs. transient, and what clearance level a promoted fact belongs to. What prompting strategy yields useful promotions at the right level? Initial approach: structured prompt asking for facts, preferences, and decisions that would be useful "next month," with an explicit step to classify each fact's minimum required clearance. Refined empirically.

12.5.3. Retrieval Relevance

Which embedding model? What chunking for daily notes across clearance levels? Re-ranking? Initial approach: nomic-embed-text via Ollama, per-paragraph chunking, hybrid keyword + vector search, filtered by clearance at query time.

12.5.4. MEMORY-k.md Budget

How large before truncation? What's the right budget relative to context window size? Initial approach: configurable per-agent per-level, default 4K tokens per level. Truncation keeps the top of the file (most curated content tends to be organized top-down).

12.5.5. Protocol State Persistence

RoundRobin and Broadcast have state (whose turn, round count, collected responses). This lives in `rooms.db` and is reconstructed on server restart.

12.5.6. FreeForm Agent-Agent Termination

For FreeForm rooms with two agents: beyond the turn limit, how do agents signal "we're done"? Initial approach: don't try to detect convergence. Use `max_turns` as the hard stop. Agents can explicitly signal completion by including a structured marker in their response.

12.5.7. Transcript Growth

Resolved by compaction and cursor-based tail reads. JSONL transcript files still grow on disk, but older messages are compacted into daily notes and excluded from later assemblies via the per-agent, per-room `compacted_offset` cursor.

12.5.8. Cost Accounting

Cloud model calls cost money. Track per-agent or per-room? Surface budgets? Initial approach: log token usage per invocation in audit; surface aggregates in UI; budget enforcement is v2.

12.5.9. OPA Policy Bootstrapping

What is the default policy shipped with the binary? It should be conservative (deny-first, require_confirmation for most agent actions) while allowing the housewife to function without extensive manual configuration. Policy is shipped as a file so the user can inspect and customize it immediately.

12.6. Glossary

Actor Any entity that participates in rooms and is subject to clearance and permissions. Either a user or an agent.

Agent An actor defined by a configuration file. Has a role, model tiers, feature flags, clearance, and permissions.

User An actor that authenticates with credentials. Holds implicit root authority.

Room A conversation container with participants, a transcript file, a clearance ceiling, and a protocol.

Protocol A behavioral contract governing how and when agents are invoked via Requests. Enforced by the dispatcher.

Request The universal invocation primitive. A dispatcher-issued instruction for an agent to produce a response. Sources: room | system | event.

Event Request A Request issued by the pub/sub system in response to an external trigger (email, calendar, webhook, schedule).

Interjection A message sent by a non-targeted actor (typically the user) while a protocol is awaiting a Request response. Queued and included in the next Request.

Project A structural grouping over rooms. Navigation and organization, not a permission boundary.

Clearance Data sensitivity level. Governs what an actor can see and what memory strata are assembled into context. Totally ordered integers; higher = more trust.

Context scope The set of memory strata available during a room interaction, determined by the room's clearance ceiling.

BLP (Bell-LaPadula) Data flow policy enforcing no read-up and no write-down without explicit approval.

ABAC (Attribute-Based Access Control) Action policy model implemented via OPA and Rego. Governs what an actor can do.

OPA (Open Policy Agent) Embedded policy engine. Evaluates permission decisions against a Rego policy file.

Permission Discrete action capability evaluated by OPA. Governs what an actor can do.

Permission effect `allow`, `require_confirmation`, or `deny`.

Config proposal A staged config or policy change awaiting user approval.

MEMORY-k.md An agent's curated long-term memory file at clearance level k. Context assembled at clearance level n includes all MEMORY-k.md where $k \leq n$.

Daily notes Per-day Markdown files at `memory/clearance-k/YYYY-MM-DD.md`. Indexed for search.

Dreaming Background consolidation pass promoting durable content from daily notes into the appropriate MEMORY-k.md.

Persistent agent Long-lived, defined by files on disk.

Ephemeral agent Task-scoped, in-memory only. Exists for parallelism and task isolation; not a security primitive.

Turn limit Maximum agent-to-agent exchanges without user participation.

MCP Model Context Protocol. Universal tool integration surface.

Explicit cross-clearance handoff Deliberate declassification of content for a lower-clearance recipient. User-confirmed.

Compaction Summarizing old room transcript messages into daily notes and advancing the `compacted_offset` cursor.

Request DAG Dependency graph of in-flight Requests. Edges model blocking relationships; enables concurrent non-blocked Request processing.

DLP (Data Loss Prevention) Structural guarantee that agents cannot exfiltrate data above the current room clearance, because higher-clearance data is absent from the assembled context.